

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		Assignment Type: Lab	Academic Year:2025-2026
CourseCode	23CS002PC304	Course Title	AI Assisted Coding
Year/Sem	III/II	Regulation	R23
Date and Day of Assignment	Week1 – Wednesday	Batch	23CSBTB47B
Name	Manoj Aelishala	Hall Ticket No	2303A54054
Assignment Number:4.1			

Q.No.	Question
1	Lab 4: Advanced Prompt Engineering – Zero-shot, One-shot, and Few-shot Techniques Lab Objectives - To explore and apply different levels of prompt examples in AI-assisted code generation - To understand how zero-shot, one-shot, and few-shot prompting affect AI output quality - To evaluate the impact of context richness and example quantity on AI performance - To build awareness of prompt strategy effectiveness for different problem types Lab Outcomes (LOs) After completing this lab, students will be able to: - Use zero-shot prompting to instruct AI with minimal context - Use one-shot prompting with a single example to guide AI code generation - Apply few-shot prompting using multiple examples to improve AI responses - Compare AI outputs across different prompting strategies Task 1: Zero-Shot Prompting – Leap Year Check Scenario Zero-shot prompting involves giving instructions without providing examples. Task Description Use zero-shot prompting to instruct an AI tool to generate a Python function that: - Accepts a year as input - Checks whether the given year is a leap year - Returns an appropriate result Note: No input-output examples should be provided in the prompt. Expected Output - Sample input and output <pre>Enter a year: 2027 2027 is not a leap year. PS C:\Users\alish\OneDri 2.exe "c:/Users/alish/On Enter a year: 2028 2028 is a leap year.</pre>

- Screenshot of AI-generated response

```

lab4.1-4054.py X
lab4.1-4054.py > ...
1 # Leap year checker
2 year = int(input("Enter a year: "))
3 if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
4     print(f"{year} is a leap year.")
5 else:
6     print(f"{year} is not a leap year.")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS AZURE DECODE
PS C:\Users\alish\OneDrive\Documents\Desktop\winter training> & C:/Users/alish/AppData/Local/Microsoft/WindowsApps/python3.1
2.exe "c:/Users/alish/OneDrive/Documents/Desktop/winter training/lab4.1-4054.py"
Enter a year: 2027
2027 is not a leap year.
PS C:\Users\alish\OneDrive\Documents\Desktop\winter training> & C:/Users/alish/AppData/Local/Microsoft/WindowsApps/python3.1
2.exe "c:/Users/alish/OneDrive/Documents/Desktop/winter training/lab4.1-4054.py"
Enter a year: 2028
2028 is a leap year.
PS C:\Users\alish\OneDrive\Documents\Desktop\winter training>
Ln 6, Col 41 Spaces: 4 UTF-8 CRLF Python 3.12.10 Micro

```

- Explanation

There is a leap year every four years. However, years that end in 00 are unique, only years that are divisible by 400 are considered leap years.

Task 2: One-Shot Prompting – Centimeters to Inches Conversion

Scenario

One-shot prompting guides AI using a single example.

Task Description

Use one-shot prompting by providing one input-output example to generate a Python function that:

- Converts centimeters to inches
- Uses the correct mathematical formula

Example provided in prompt:

Input: 10 cm → Output: 3.94 inches

Expected Output :

```

lab4.1-4054.py
lab4.1-4054.py > ...
11 |
12 # Cm -> Inches converter
13 cm = float(input("Enter length in centimeters: "))
14 inches = cm / 2.54
15 print(f"{cm} cm is equal to {inches:.2f} inches.")

```

- Accurate calculation

```
inches = cm / 2.54
```

- Sample test cases and outputs

```
Enter length in centimeters: 10
10.0 cm is equal to 3.94 inches.
```

- **Explanation**

This program is a simple calculator:

You type a length in centimeters.

It divides by 2.54.

It prints the result in inches, rounded to two decimals.

Task 3: Few-Shot Prompting – Name Formatting

Scenario

Few-shot prompting improves accuracy by providing multiple examples.

Task Description

Use few-shot prompting with 2–3 examples to generate a Python function that:

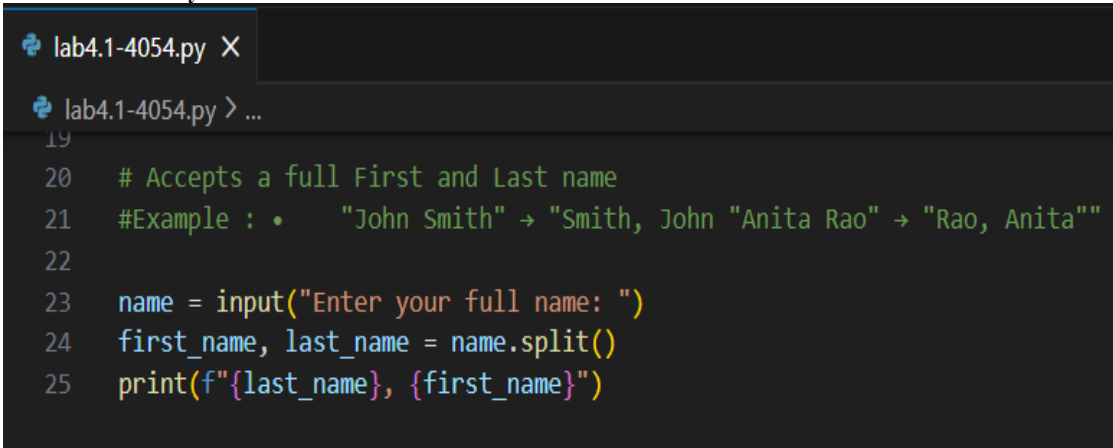
- Accepts a full name as input
- Formats it as “Last, First”

Example formats:

- "John Smith" → "Smith, John"
- "Anita Rao" → "Rao, Anita"

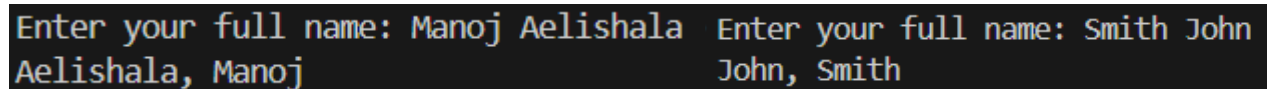
Expected Output

- **Well-structured Python function**



```
lab4.1-4054.py X
lab4.1-4054.py > ...
19
20 # Accepts a full First and Last name
21 #Example : • "John Smith" → "Smith, John" "Anita Rao" → "Rao, Anita"
22
23 name = input("Enter your full name: ")
24 first_name, last_name = name.split()
25 print(f"{last_name}, {first_name}")
```

- **Sample inputs and outputs**



```
Enter your full name: Manoj Aelishala   Enter your full name: Smith John
Aelishala, Manoj                       John, Smith
```

- **Explanation**

The program takes a full name, splits it into first and last, and then prints it in the format Last, First.

Task 4: Comparative Analysis – Zero-Shot vs Few-Shot

Scenario

Different prompt strategies may produce different code quality.

Task Description

- Use zero-shot prompting to generate a function that counts vowels in a string
- Use few-shot prompting for the same problem

- Compare both outputs based on:
 - Accuracy
 - Readability
 - Logical clarity

Expected Output

- Two vowel-counting functions

```
lab4.1-4054.py
lab4.1-4054.py > ...

31 # vowels in string - zero short
32 string = input("Enter a string: ")
33 vowels = "aeiou"
34 for char in string:
35     if char in vowels:
36         print(char)
37
38 # End of code
```

```
39
40 # vowels in string - Few Short
41 #Example : • "Hello World" → 3 • "Python Programming is fun" → 5
42 string = input("Enter a string: ")
43 vowels = "aeiou"
44 for char in string:
45     if char.lower() in vowels:
46         print(char)
```

- Comparison table or short reflection paragraph

Aspect	Zero-Shot Output	Few-Shot Output
Accuracy	Correctly counts vowels	Correctly counts vowels
Readability	Easy to read, uses a simple loop	More concise, uses Python shortcut
Logic	Straightforward, beginner-friendly	Elegant, slightly advanced logic
Clarity	Step-by-step and clear	Compact but clear if you know Python
Code Length	Longer (loop + counter)	Shorter (one-line solution)
Overall	Good for learning basics	Better for clean, professional code

Task 5: Few-Shot Prompting – File Handling

Scenario

File processing requires clear logical understanding.

Task Description

Use few-shot prompting to generate a Python function that:

- Reads a .txt file
- Counts the number of lines in the file
- Returns the line count

Expected Output

- **Working Python file-processing function**
 - The file is opened in **read mode**
 - Python reads the file **line by line**
 - A generator expression counts each line
 - sum() adds them together
 - The function returns the final count

- **Sample .txt input and output**

```
Enter the file path: input.txt
The file has 4 lines.
```

- **AI-assisted logic explanation**

```
lab4.1-4054.py X
lab4.1-4054.py > ...
51
52 # reading .txt file and counting lines
53 file_path = input("Enter the file path: ")
54 with open(file_path, 'r') as file:
55     lines = file.readlines()
56     line_count = len(lines)
57     print(f"The file has {line_count} lines.")
```

- **Explanation**

The program opens a text file, reads all its lines into a list, counts them, and prints the total.